Lab 06 – Functions and Modular Programming
CLO: CLO2

Learning Objectives
After completing this lab, students will be able to:

- Define and call functions
- Use parameters and return values
- Apply local/global variables
- Build reusable modules
- Organize code into smaller components
- Improve code readability and maintainability

(0.25 point)
Exercise 1 – Factorial Calculator

Problem Statement
Write a function to calculate factorial.
Example:

$$5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$$

(0.25 point)
Exercise 2 – Fibonacci Generator

Problem Statement
Generate Fibonacci sequence:
Rule:

$$F(n) = F(n-1) + F(n-2)$$

(0.5 point)
Exercise 3 – Statistics Calculator
Problem Statement
Create functions to calculate:

- Sum
- Average
- Maximum
- Minimum

from a list of numbers.

(0.5 point)

Exercise 4 – Utility Module

Problem Statement

Separate functions into modules.

Project Structure:

```
project/
│
├── main.py
├── math_utils.py
└── string_utils.py
```

(0.5 point)

Exercise 4 – Menu-Driven Utility System

Requirement:

Build:

```
===== UTILITY SYSTEM =====
1. Factorial
2. Fibonacci
3. Statistics
4. String Utilities
5. Exit
```

Each feature must be implemented as separate functions.

(1 point)

Exercise 5 – Password Validator

Requirement

Create function: validate_password(password)

Rules:

- minimum 8 chars
- uppercase required
- lowercase required
- number required
- special character required

(1 point)

Exercise 6 – Student Grade Management System

Requirements

Functions:

- add_student()
- calculate_average()
- find_top_student()
- search_student()

Store data using:

- dictionary
- list

(2 points)
Exercise 7 – Build Your Own Python Package

Problem Statement
Build a reusable Python package called: mypackage
The package should contain:

- mathematical utilities
- text utilities
- reusable functions

Structure:

```
mypackage/
│
├── __init__.py
├── math_tools.py
├── text_tools.py
└── main.py
```

Goal
Students learn:

- package structure
- imports
- reusable architecture

(2 points)
Exercise 8 – Functional Programming Style

Learning Objectives

Students learn:

- lambda functions
- map()
- filter()
- functional programming style
- concise coding

Challenge A – Word Processing Pipeline

Requirements:

- split words
- convert uppercase
- keep words length > 5

Example:

- Input: "Python is powerful and flexible"
- Expected Output: ['PYTHON', 'POWERFUL', 'FLEXIBLE']

Challenge B – Student Grade Processing

Requirements:

- keep students score >= 50
- add bonus +5
- sort descending

(2 points)

Exercise 9 – Benchmark Performance

Learning Objectives

Students learn:

- performance measurement
- runtime analysis
- algorithm comparison
- optimization thinking

Problem Statement

Compare runtime of:

1. iterative factorial
2. recursive factorial

Challenge A – Benchmark Fibonacci

Compare:

- iterative Fibonacci
- recursive Fibonacci

Advanced Discussion Topics

Topic	Concepts
iterative vs recursive	stack overhead
memoization	dynamic programming
benchmarking	performance engineering
runtime analysis	Big-O thinking